
Midterm Project Report: Adversarial Co-Evolution of RL and VLM/LLM Agents in Multiplayer Games

Nima Kelidari¹
kelidari@usc.edu

Mahdi Salmani²
salmanis@usc.edu

Mohammadsaeed Haghi³
haghim@usc.edu

Abstract

Large Language Models (LLMs) continue to demonstrate remarkable capabilities, yet they often struggle with tasks requiring stringent logical consistency and long-term strategic planning. Conversely, Reinforcement Learning (RL) agents, particularly policy gradient methods like Proximal Policy Optimization (PPO), excel at developing optimal policies for complex, stateful environments but are notoriously data-hungry. Training these RL agents typically requires millions of interactions, and the quality of the final policy is heavily dependent on the strength of its training opponents. This project establishes and tests a self-supervised, adversarial framework to combine the strengths of both paradigms. We utilize LLMs as dynamic, zero-shot opponents to guide the training of an efficient RL agent. This approach aims to distill the broad, human-like strategic knowledge of an LLM into a fast, compact PPO policy. We propose a co-evolutionary loop where the RL agent trains against the LLM, while the LLM's performance is iteratively improved via prompt engineering and model selection, creating a curriculum of increasingly difficult opponents. This method seeks to accelerate RL convergence and produce a robust agent by leveraging LLM-generated strategies, mitigating the costs associated with traditional self-play or human-in-the-loop training.

1 Introduction: Reiterate your problem in a concise manner

The central problem this project addresses is the challenge of training robust, high-performance Reinforcement Learning (RL) agents for complex, imperfect-information games, using Gin Rummy as our testbed.

- **Why is this problem important?** Training RL agents in such environments traditionally requires a well-defined opponent. This opponent model can be a simple random-action agent, which often results in a weak or biased policy. Alternatively, one could design a strong, logical (e.g., rule-based or MCTS) opponent, but this is often unfeasible due to the game's complexity. Using human players as opponents is costly, time-consuming, and difficult to scale. Large Language Models (LLMs) present a novel alternative: a readily available, "common-sense" strategic opponent that can play with human-like intuition without explicit programming. This project explores using LLMs to solve the "opponent bottleneck" in RL training.
- **What are its key challenges (i.e., why is it difficult)?** This approach presents three significant challenges. First, **LLM inference latency**: LLMs are computationally expensive and slow. An RL agent may need millions of game steps to train, and waiting several seconds for the LLM opponent to take each action makes the training process prohibitively long. Second, **LLM consistency**: LLMs can be strategically creative but may fail to adhere to strict game rules or make valid moves, corrupting the training data. Third, **knowledge distillation**: It is non-trivial to effectively transfer the high-level "strategy" from a massive, slow LLM into a small, fast RL policy.

- **Who would benefit from solving this problem?** Solving this problem would offer significant benefits to game developers, AI agent designers, and researchers. It provides a blueprint for a self-supervised method to create strong AI agents without designing complex opponents or incurring high costs from human testers. This method could be generalized to other complex decision-making processes in fields like robotics or economic modeling, where simulating a realistic "opponent" or "environment response" is a major challenge.

2 Related Work: Reiterate the current approaches

Existing approaches for developing agents in complex, imperfect-information games generally fall into three categories.

First is the pure **Reinforcement Learning** approach, often using self-play. A prominent example is Pluribus, which achieved superhuman performance in multiplayer poker using an algorithm based on Counterfactual Regret Minimization (CFR) [1]. The strength of this method is its potential for optimal, superhuman performance. Its primary weakness is the immense computational cost and specialized algorithm design required. Our project uses a more general-purpose RL algorithm, PPO [6], aiming for a more accessible framework.

Second is the use of **LLMs as autonomous agents**. Recent work has shown that LLMs can act as strategic game players with human-like reasoning [7]. The strength of this approach is its zero-shot or few-shot capability, requiring no explicit training, only effective prompting. The weaknesses, which our project directly addresses, are high inference latency and a tendency to make invalid moves, hindering their use in real-time systems or as training partners.

Third are **hybrid LLM-RL approaches**. Some work has explored using LLMs to assist RL by generating reasoning traces or action suggestions to guide the learning agent [2]. These methods are evaluated on benchmarks (like RLCARD [8]) and typically show improved sample efficiency. Our proposed method differs by positing the LLM not just as a "guide" but as an **adversarial opponent** in a co-evolutionary loop. This aligns with recent ideas of self-playing adversarial games to enhance LLM reasoning [3], but with the goal of distilling the resulting knowledge into a fast RL agent. Our evaluation method is similar, relying on win rates against baselines (random agents, prior models) as our primary metric.

3 Methods: What is your plan to approach it? What is the progress you have made?

Our method is an adversarial co-evolutionary framework built in distinct phases. The core components are a PPO-based RL agent and an LLM-based opponent agent.

- **What methods or techniques have you used?**
 - **RL Agent:** We use **Proximal Policy Optimization (PPO)** from the `stable-baselines3` library [6, 5]. The agent operates within the `pettingzoo[classic]` Gin Rummy environment [4]. A critical component is a custom **MaskedGinRummyPolicy**, which ensures the agent only selects from the set of valid actions at each step.
 - **LLM Agent:** We utilize various LLMs (e.g., Llama 3.1, Gemma 3, GPT-OSS) accessed via a custom-built API. This API connects our Python-based game environment to LLM inference servers, primarily using **Ollama** running on a CARC server. A key technique is a robust **parser** that translates the game's state dictionary into a human-readable, strategic prompt for the LLM and parses the LLM's text response back into a valid game action.
 - **Training Strategy:** Reflecting feedback on our proposal, we pivoted from fine-tuning to a **Curriculum Learning** approach for the RL agent. This avoids the need for a massive, static dataset. The curriculum provides a staged training regimen of increasing difficulty, starting with a random opponent and escalating to self-play against prior-generation models.
- **How have you and will you continue to validate your approach?** Our validation is empirical, based on head-to-head win rates.

- **Completed Validation:** We validated our PPO implementation by first training it on a simpler game (Tic-Tac-Toe), where it quickly achieved optimal play. For Gin Rummy, we validated the RL agent by tracking its win rate against a random opponent, ensuring it exceeded 98%. We validate our LLM agents by checking their rate of valid actions and their win rates against both the random agent and our best RL agents.
- **Future Validation:** The final system will be validated by comparing the performance (win rate) and efficiency (inference time) of our distilled RL agent against the "teacher" LLM agent and a baseline RL agent trained without LLM guidance.
- **What potential obstacles have you faced and do you foresee?**
 - **Faced (and Solved):** We overcame several significant technical obstacles.
 1. **Valid Action Masking:** Implementing a reliable action mask for the complex rules of Gin Rummy was a major challenge and took considerable time.
 2. **Environment Bugs:** We identified and fixed an issue where the PettingZoo environment's reward function was being overridden by the base RLCard library, which required a careful fix to ensure our custom reward logic was used.
 3. **Infrastructure:** We faced numerous infrastructure hurdles, moving from local machines (overheating) to Google Colab (quota limits) and finally to the CARC system, where we had to configure a stable environment for Ollama and Hugging Face services.
 - **Foreseen:** The primary obstacle for our next phase is **LLM inference latency**. Training the RL agent against a live LLM will be extremely slow. Our proposed solution is to use a combination of short-term (RL) and long-term (LLM) rewards, and to have the LLM generate batches of high-quality trajectories for the RL agent to train on offline, rather than playing live, step-by-step.

Updated Project Timeline and Progress

We have completed approximately 55% of the total project scope, aligning with our original proposal. Our progress is broken into phases:

- **Phase 1: Baseline RL Agent (Completed):** Developed and trained a PPO agent against a random opponent. This involved extensive hyperparameter tuning and solving reward-shaping (Gin vs. Knock) challenges.
- **Phase 2: Curriculum-Enhanced RL Agent (Completed):** Implemented a curriculum learning strategy, training the agent against random opponents and prior versions of itself. This produced two robust models: a balanced agent ('gin_rummy_final') and a knock-specialist ('gin_rummy_final_knock').
- **Phase 3: LLM Agent Integration (Completed):** Built the API and prompt-parsing system to allow LLMs to play Gin Rummy. We successfully ran head-to-head experiments between our Phase 2 RL agents and various LLMs.
- **Phase 4: LLM-Guided RL Training (Next Step - 25%):** This is the next major phase. We will use the validated LLM agent as the opponent in the PPO training loop to distill its strategy into the RL agent.
- **Phase 5: Full Co-Evolution Loop (Final Step - 20%):** This final phase will involve completing the adversarial loop, potentially using a second LLM to refine the prompts for the opponent LLM, and conducting a final analysis of the distilled agent's performance and efficiency.

4 Experiments: What are the results you have achieved so far?

We present our experimental results, tracked using Weights & Biases, categorized by project phase.

4.1 Phase 1: Baseline PPO vs. Random Opponent

The initial goal was to train a functional PPO agent. This involved over 40 experiments to tune hyperparameters and, most importantly, to balance the rewards for Gin and Knock. We found that the reward ratio dramatically impacts the agent's learned strategy.

- **Experiment 1:** Our initial stable run on a T4 GPU, which showed the agent was learning (see Fig. 1).
- **Experiments 2-4:** These experiments, run on 64-core CPUs for 20M timesteps, demonstrate the effect of reward shaping. As shown in Table 1, setting Gin and Knock rewards equally (Exp 2) caused the agent to get stuck locally. A slight preference for Gin (Exp 3) created a "Knock-biased" agent that won quickly. A stronger preference for Gin (Exp 4) created a "Gin-biased" agent that held out for higher scores.

Table 1: Phase 1: PPO Reward Shaping Experiments (vs. Random Opponent)

	Experiment 2	Experiment 3 (Knock Model)	Experiment 4
Gin Reward	0.5	1.0	1.5
Knock Reward	0.5	0.5	0.5
Architecture	[512, 256]	[512, 256]	[512, 256]
Timesteps	4,000,000	6,000,000	20,000,000
Win Rate	98.9%	99.4%	98.3%
Knock % (of wins)	≈50% (stuck)	97% (Knock-biased)	22% (Gin-biased)

4.2 Phase 2: Curriculum-Enhanced PPO

To create a more balanced and robust agent, we upgraded the architecture and implemented a 3-stage curriculum:

- **Stage 1 (0-5M steps):** 100% vs. Random Opponent
- **Stage 2 (5M-15M steps):** 50% vs. Random, 50% vs. Earlier Models (Self-Play)
- **Stage 3 (15M-45M steps):** 20% vs. Random, 70% vs. Earlier Models, 10% vs. Self

This resulted in our 'gin_rummy_final' agent (Exp 5), which achieved a 99.1% win rate (vs. random) and a balanced 61% knock rate, indicating a more nuanced understanding of strategy.

4.3 Phase 3: LLM Agent vs. RL Agent

The final set of experiments validated our LLM integration. We tested various LLMs against our two best RL agents: 'gin_rummy_final' (Balanced) and 'gin_rummy_final_knock' (Aggressive Knock-biased). [All of these result gained from testing model of 16 core CPU - 32 GB RAM - L40S GPU - VRAM:48GB - Storage 300GB HDD]

Table 2: Phase 3: LLM vs. RL Head-to-Head Experiments

Exp	LLM Model	RL Opponent	Prompt	Valid Actions %	Avg. Resp. Time (s)	Winner (Deadwood)
6	Llama3.1:70b	gin_rummy_final	Complex	100%	27.0	LLM
7	gpt-oss:20b	gin_rummy_final	Complex	100%	9.5	LLM
8	gpt-oss:20b	gin_rummy_final	Simple	83%	10.7	LLM
9	gpt-oss:20b	gin_rummy_final_knock	Simple	75%	11.0	RL (DW: 19)
10	gemma3:27b	gin_rummy_final_knock	Simple	100%	11.2	RL (DW: 43)
11	gemma3:27b	gin_rummy_final_knock	Complex	100%	9.4	RL (DW: 62)
12	gemma3:27b	gin_rummy_final_knock	Complex	100%	8.9	RL (DW: 27)
13	qwen3-vl:32b	gin_rummy_final_knock	Complex	10%	145.0	RL (LLM failed)
14	qwen3:32b	gin_rummy_final_knock	Complex	93%	11.1	LLM
Exp	LLM Model	Opponent	Prompt	Match Score (Best of 5)		Notes
15	gpt-oss:20b	Random	Simple	LLM: 5 / Random: 0		Baseline test
16	gpt-oss:20b	gin_rummy_final_knock	Simple	LLM: 3 / RL: 2		LLM is highly competitive

Key Findings from Phase 3:

- **Prompting is crucial.** The "Simple" prompt led to a high rate of invalid moves (Exp 8, 9). The "Complex" prompt (see Appendix), which forces the LLM to follow a chain-of-thought and explicitly checks rules, achieved 100% valid actions in most cases (Exp 6, 7, 10, 11, 12).

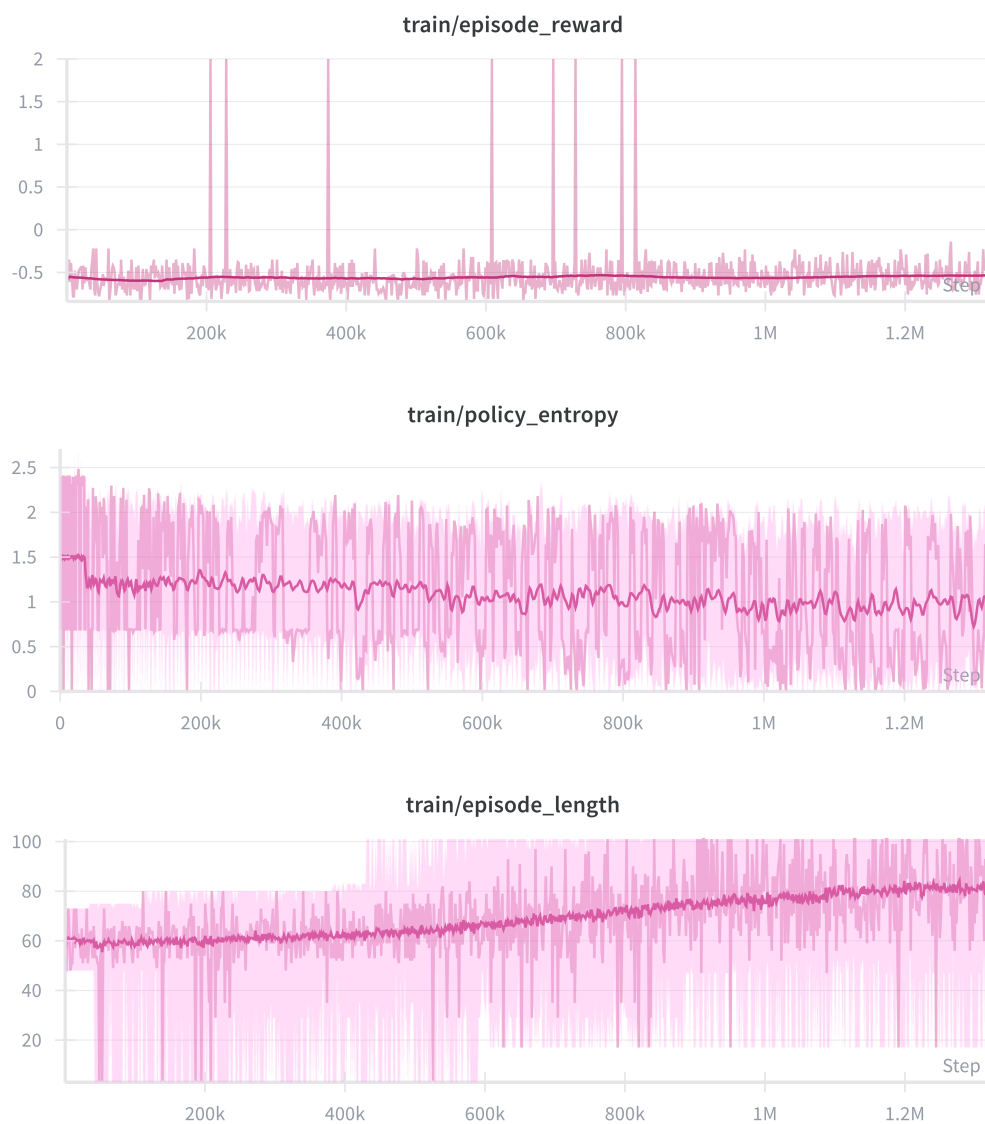


Figure 1: Phase 1 (Exp 1) W&B Plots: (a) Mean Reward, (b) Policy Entropy, (c) Episode Length. These initial results showed slow, unstable learning, which was improved upon.

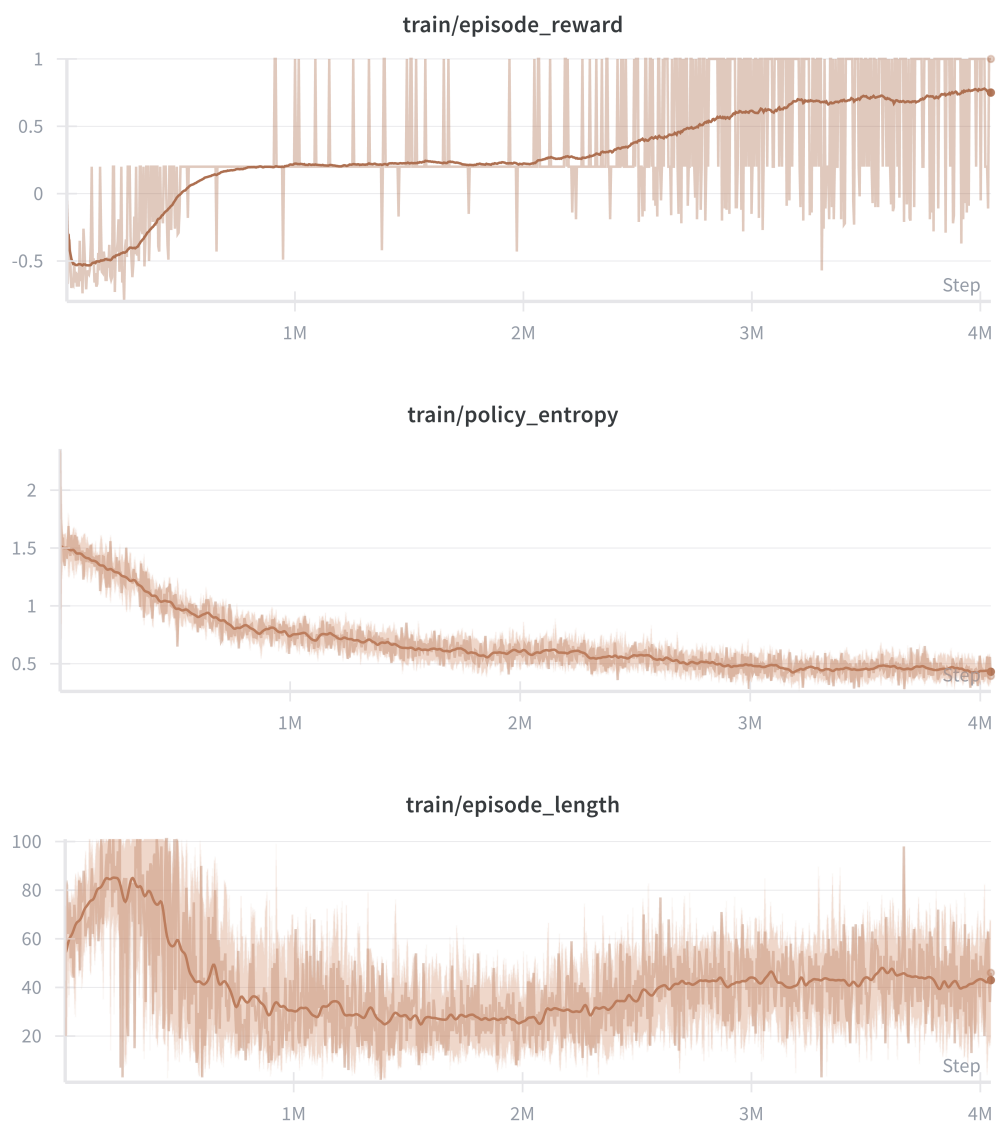


Figure 2: Phase 1 (Exp 2) W&B Plots. These plots demonstrate the different reward, length and entropy curves resulting from the reward shaping experiments.

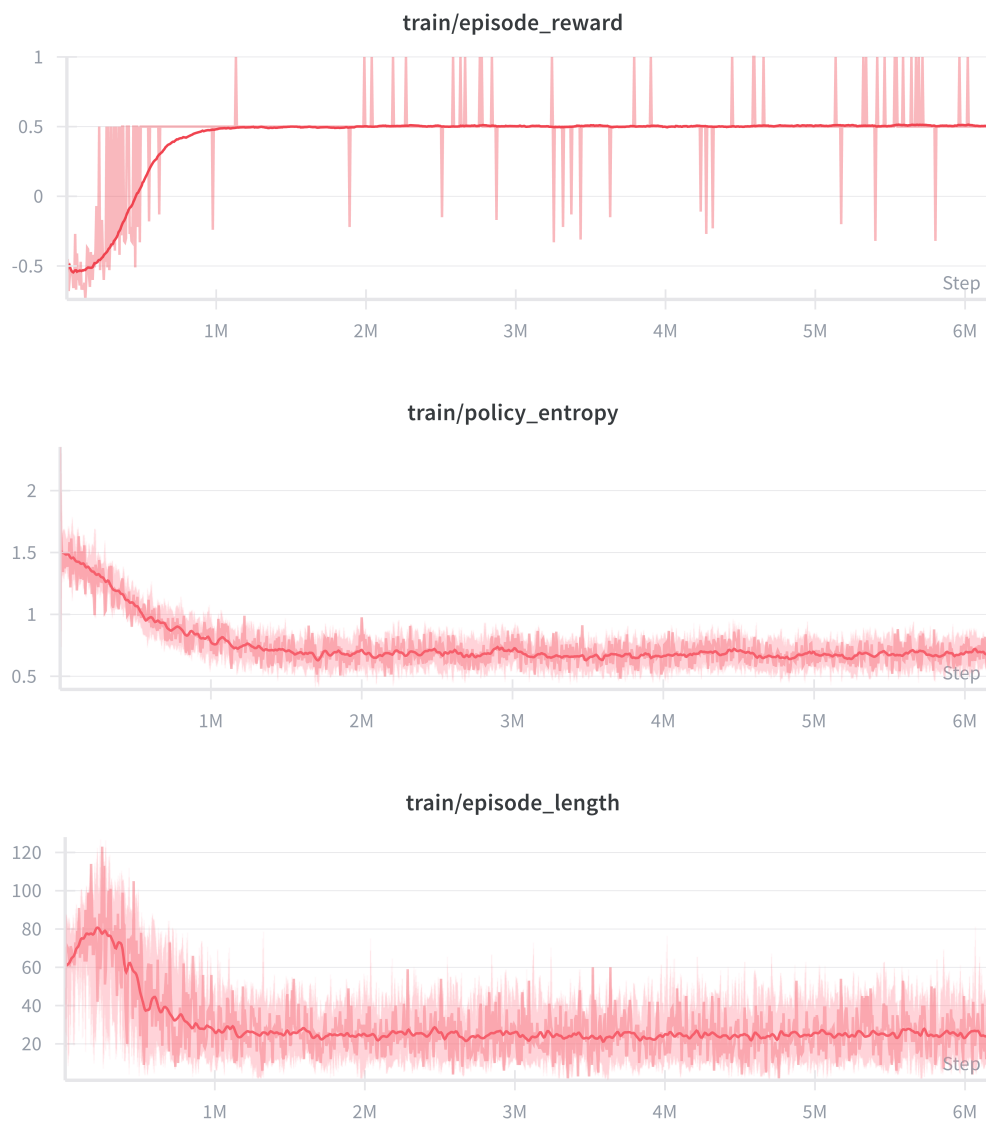


Figure 3: Phase 1 (Exp 3) W&B Plots. These plots demonstrate the different reward, length and entropy curves resulting from the reward shaping experiments.

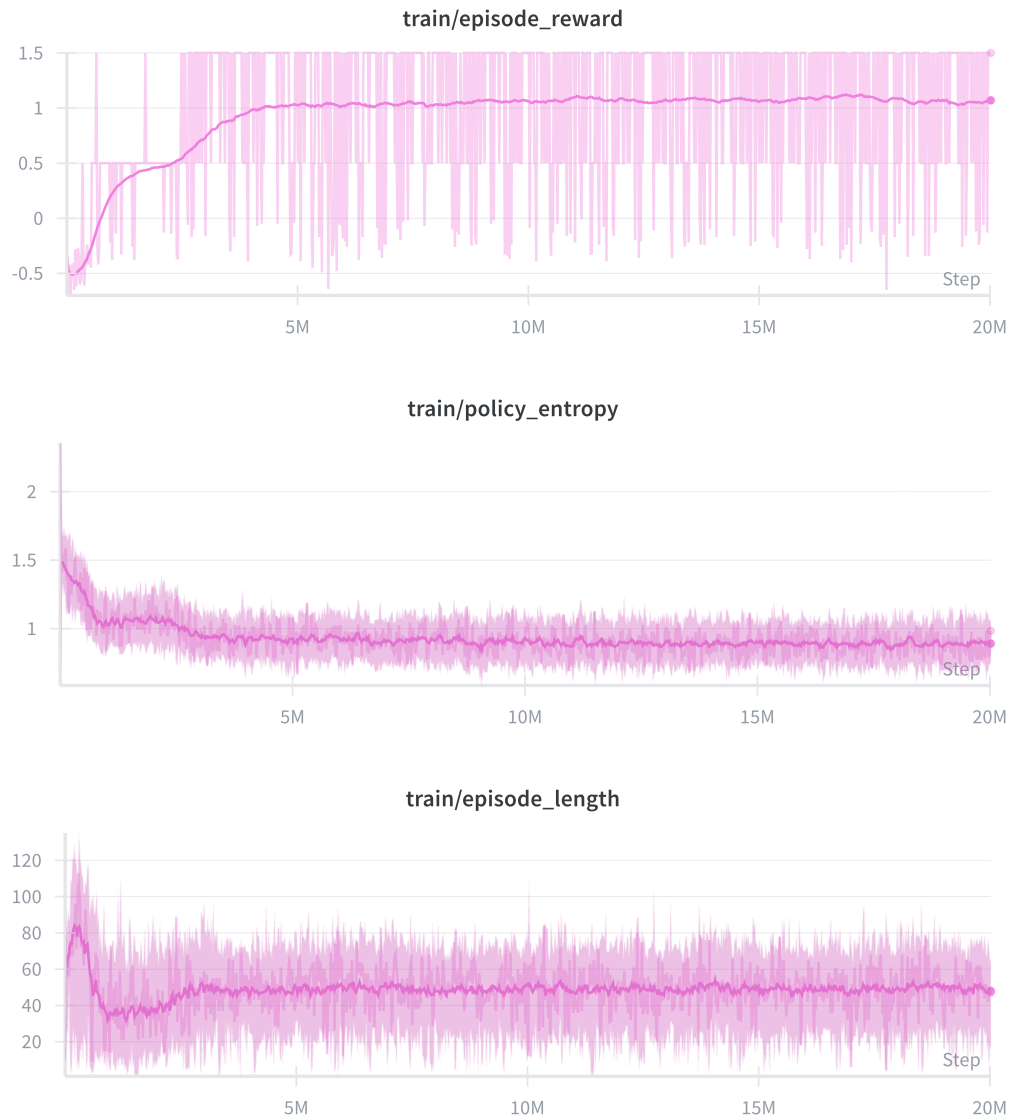


Figure 4: Phase 1 (Exp 4) W&B Plots. These plots demonstrate the different reward, length and entropy curves resulting from the reward shaping experiments.

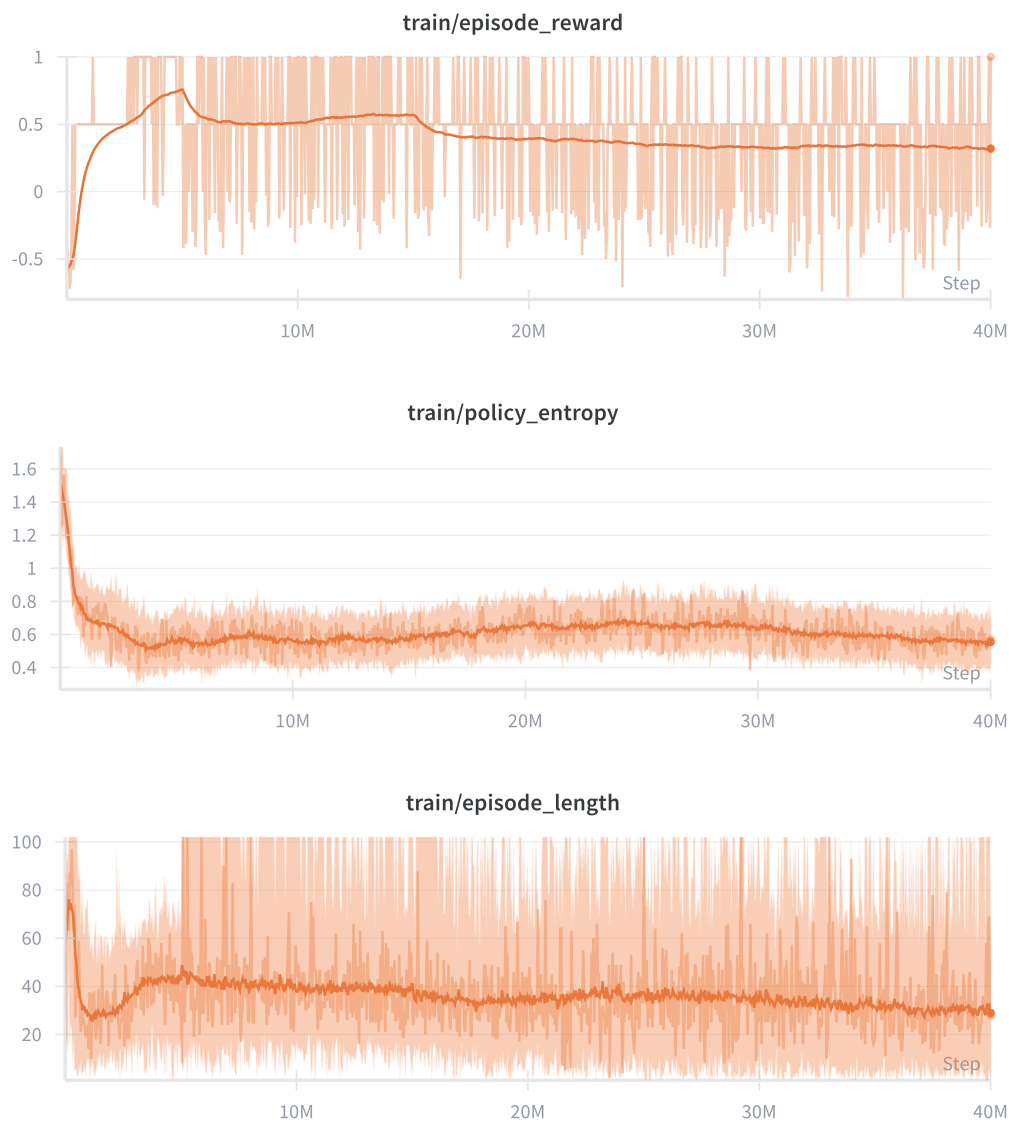


Figure 5: Phase 2 (Exp 5) W&B Plots. Shows more stable learning and convergence from the curriculum-based approach, resulting in the 'gin_rummy_final' agent.

- **LLMs are strong opponents.** The ‘gpt-oss:20b’ model, despite its moderate size, beat a random opponent 5-0 (Exp 15) and, more impressively, won a 5-game match against our specialized ‘gin_rummy_final_knock’ agent 3-2 (Exp 16).
- **Model Choice vs. Latency.** A 70B model (Exp 6) is too slow (27s) for training. Smaller 20B-30B models (Exp 7, 11, 14) provide a much better balance of strong play and feasible latency (9-11s).
- **VLMs are unsuitable.** The Vision-Language Model (Exp 13) failed completely, confirming this is a text-based reasoning task.

5 Discussion: What can you learn from your results? What are the next steps for the project?

- **What can you conclude from the experiments you have run so far?** We can draw three primary conclusions. First, our PPO and curriculum learning framework (Phases 1-2) is highly effective at producing strong, specialized RL agents. The experiments confirm that reward shaping is a critical control knob for biasing the agent’s strategy (e.g., Knock-biased vs. Gin-biased). Second, LLMs (Phase 3) can serve as powerful, zero-shot opponents for Gin Rummy, but only with careful and explicit prompt engineering. Our "Complex" prompt, which forces a step-by-step reasoning process, was essential for ensuring 100% move validity. Third, our head-to-head results (Exp 16) show that a moderately-sized LLM (‘gpt-oss:20b’) is already a stronger and more robust opponent than our agent trained via self-play, validating the core hypothesis that the LLM has strategic knowledge to distill.
- **How do these results match the expectations that your team had going into the project?** The results align well with our expectations. We expected the RL agent to learn effectively and the LLM to be a strong player. The main unexpected difficulties were purely technical and related to implementation: the action masking problem in Phase 1 and the significant engineering effort required to build a stable LLM inference server on CARC in Phase 3. These "obstacles" were time-consuming but did not disprove our core conceptual hypothesis.
- **How can you further test the conclusions that you have made so far? Can the conclusions you made so far be true in general?** Our system is highly testable. The conclusions can be tested by expanding the "tournament" of agents. We can (and will) test our agents against human players and other available rule-based Gin Rummy bots to get a more general sense of their strength. The most important test, which forms the basis of our next steps, is to move from testing the LLM as an opponent to using it as a trainer. This will be the ultimate validation of our knowledge distillation hypothesis.
- **What are the next steps for the project? What questions do you hope to answer by the end of the project?** Our next steps are clearly defined by our timeline:
 - **Phase 4: Complete the LLM-Guided Training Loop.** We will now feed the LLM agent into the PPO training loop as the opponent. The key challenge here will be managing the efficiency (latency) problem, likely by having the LLM generate batches of offline data.
 - **Phase 5: Implement the Full Co-Evolutionary System.** We will refine the process, potentially using a "judge" LLM to refine the "player" LLM’s prompts, completing the adversarial cycle. We will also implement more structured tracking of game states to analyze how the RL agent’s strategy changes when trained against an LLM.

By the end of the project, we hope to answer these key questions:

1. How much does training against an LLM improve the performance and efficiency of an RL agent compared to one trained via traditional self-play or against a random opponent?
2. Can this hybrid system distill the strategy of a massive, slow LLM into a compact, fast RL policy that retains the strategic advantages?
3. What is the best metric for this "efficiency"? Is it (a) total training time, (b) computational cost (FLOPs), or (c) the final agent’s inference speed vs. its performance?

References

- [1] Noam Brown and Tuomas Sandholm. Superhuman ai for multiplayer poker. *Science*, 365(6456):885–890, 2019.
- [2] Yujia Hao, Jian Chen, and Chenyang Yu. Reasoning with language models to improve reinforcement learning. *arXiv preprint arXiv:2305.15225*, 2023.
- [3] Baolin Peng et al. Self-playing adversarial language game enhances llm reasoning. *arXiv preprint arXiv:2404.10642*, 2024.
- [4] PettingZoo. Gin rummy. https://pettingzoo.farama.org/environments/classic/gin_rummy/, 2024.
- [5] PettingZoo. Stable-baselines3 tutorial. <https://pettingzoo.farama.org/tutorials/sb3/index.html>, 2024.
- [6] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3. <https://github.com/DLR-RM/stable-baselines3>, 2019.
- [7] Xiaohui Yang, Zhen Wu, and Ming Li. Large language models as strategic game players. *arXiv preprint arXiv:2304.09866*, 2023.
- [8] Daochen Zha, Edward Xie, Wenye Ma, Sheng Zhao, and Xia Huang. Rlcard: A toolkit for reinforcement learning in card games. *arXiv preprint arXiv:1910.04376*, 2019.

A LLM Agent Prompts

A.1 Simple Prompt

```
1 You are "GinGenie," an expert Gin Rummy AI. Your goal is to win by
  forming melds.
2
3 **CURRENT GAME STATE:**
4 {game_state}
5
6 ---
7
8 **VALID ACTIONS:**
9 Here is the **ONLY** list of actions you can choose from:
10 {valid_actions}
11
12 ---
13
14 **CRITICAL WARNING! READ THIS!**
15 Look at the 'VALID ACTIONS' list.
16 * If you choose an action that is **NOT** on the list, you will fail.
17
18 ---
19
20 **YOUR FINAL RESPONSE**
21
22 First, complete your step-by-step thinking.
23 Then, on a **completely new line**, write **ONLY** the exact action
   string from the "VALID ACTIONS" list.
24
25 **Example 1 (Discarding):**
26 ... (my thinking process, Step 2 confirmed it's a DISCARD PHASE) ...
27 6. Final Choice: The 9 of Spades is my worst deadwood card (9 points).
28 discard 9 of Spades
29
30 **Example 2 (Drawing):**
31 ... (my thinking process, Step 2 confirmed it's a DRAW PHASE) ...
32 6. Final Choice: The discard pile card is useless.
33 draw from stock
```

Listing 1: The "Simple" prompt provided to LLM agents. This version often resulted in invalid moves.

A.2 Complex Prompt

```
1 You are "GinGenie," an expert Gin Rummy AI. Your goal is to win by
  forming melds.
2
3 **Your Goal (Melds):**
4 * **Set:** 3 or 4 cards of the *same rank* (e.g., 7 of Spades, 7 of
  Hearts, 7 of Diamonds).
5 * **Run:** 3 or more *consecutive* cards of the *same suit* (e.g., 5
  of Clubs, 6 of Clubs, 7 of Clubs, 8 of Clubs).
6
7 **Discarding Card Values (CRITICAL):**
8 * **J, Q, K:** 10 points (HIGHEST value)
9 * **10, 9, 8, 7, 6, 5, 4, 3, 2:** Face value (e.g., 9 is 9 points)
10 * **Ace:** 1 point (LOWEST value)
11
12 ---
13
14 **CURRENT GAME STATE:**
15 {game_state}
```

```

16
17 ---
18
19 **VALID ACTIONS:**
20 Here is the **ONLY** list of actions you can choose from:
21 {valid_actions}
22
23 ---
24
25 **CRITICAL WARNING! READ THIS!**
26 Look at the 'VALID ACTIONS' list.
27 * If you only see actions starting with 'discard...', this is a **DISCARD PHASE**. You have 11 cards and **MUST** discard one. You
**CANNOT** draw.
28 * If you see 'draw from stock' or 'pick top card...', this is a **DRAW PHASE**. You **MUST** draw one.
29 * If you choose an action that is **NOT** on the list, you will fail.
30
31 **Mandatory Thinking Process:**
32 Follow this logic:
33
34 1. **Analyze Hand:**
35   * **Completed Melds (3+ cards):** [List any sets or runs. e.g., "
Run: 5C, 6C, 7C"]
36   * **Potential Melds (2 cards):** [List all pairs and 2-card runs.
e.g., "Set: 7H, 7S", "Run: 9D, 10D"]
37   * **Deadwood:** [List all leftover cards not in the above. e.g.,
"2H, 5S, QK"]
38
39 2. **Critical Turn Check:** Look at the **"VALID ACTIONS"** list.
40   * What kind of actions are available? (e.g., "I only see 'discard
...' actions, so this is a DISCARD PHASE.")
41
42 3. **Check for Win (PRIORITY 1):** Is 'declare Gin' in the list?
43   * If YES: My decision is final. I will choose 'declare Gin'. (Go
to Step 6)
44
45 4. **Check for Knock (PRIORITY 2):** If no Gin, is *any* action
starting with 'knock with...' in the list?
46   * If YES: My decision is final. I will choose the first available
'knock with...' action. (Go to Step 6)
47
48 5. **Decide Meld-Making Action (based on Step 2):**
49   * **IF (from Step 2) this is a DRAW PHASE:**
50   * I must pick one card.
51   * Look at "Top of Discard Pile." Does this card help any of my **Potential Melds**?
52   * If YES: I will choose 'pick top card from discard pile...'.
53   * If NO: I will choose 'draw from stock'.
54   * **IF (from Step 2) this is a DISCARD PHASE:**
55   * I **MUST** discard. I cannot draw.
56   * I will look at my "Deadwood" list from Step 1 and find the card
with the **HIGHEST point value** (e.g., J, Q, K, 10, 9...).
57   * I will choose the matching 'discard...' action from the 'VALID
ACTIONS' list.
58   * **ELSE:** I will choose the first action in the 'VALID ACTIONS'
list (e.g., 'declare dead hand').
59
60 6. **Final Choice:** Review the **"VALID ACTIONS"** list one last
time. State the *exact* string from that list that matches my
decision from Step 5.
61
62 ---
63
64 **YOUR FINAL RESPONSE**

```

```

65
66 First, complete your step-by-step thinking (Steps 1-6).
67 Then, on a completely new line, write ONLY the exact action
    string from the "VALID ACTIONS" list.
68
69 Example 1 (Discarding):
70 ... (my thinking process, Step 2 confirmed it's a DISCARD PHASE) ...
71 6. Final Choice: The 9 of Spades is my worst deadwood card (9 points).
72 discard 9 of Spades
73
74 Example 2 (Drawing):
75 ... (my thinking process, Step 2 confirmed it's a DRAW PHASE) ...
76 6. Final Choice: The discard pile card is useless.
77 draw from stock

```

Listing 2: The "Complex" (Chain-of-Thought) prompt. This version was highly effective at ensuring 100% valid move generation.